Integrated Boarding House Management System (IBHMS).

This system will centralize tenant records, room availability, payment tracking, maintenance handling, and security logs into a unified digital platform. It will provide the management with real-time access to occupancy information, financial summaries, and tenant histories. The system will also support basic online accessibility, enabling potential tenants to inquire and view room availability digitally.

The IBHMS aims to improve operational efficiency, reduce human error, enhance tenant satisfaction, and provide the business with the digital presence needed to remain competitive in its area.

Subsystem Objectives and Requirements

A. Tenant Management Subsystem

The current reliance on manual filing systems, such as physical folders and disconnected spreadsheets, creates significant operational risks. Tenant data is often fragmented, leading to lost contact details, inability to quickly verify identity during emergencies, and overlooked lease expirations that result in revenue loss. There is no single "source of truth" to view a tenant's complete history. This subsystem aims to solve these issues by digitizing the entire tenant lifecycle. The scope explicitly includes the digital registration of tenant profiles, secure uploading and storage of ID documents, and the end-to-end management of lease contracts—covering creation, automated renewal notifications, and termination workflows. While the system manages the data, it will not perform external credit scoring or background checks, which remain manual pre-requisites.

- Maintain a digital database of tenant profiles, contracts, and room assignments.
- Automate tenant registration, move-in, and move-out processes.
- Track lease durations, renewal dates, and status updates.
- Allow managers to quickly search, update, and monitor tenant information.

B. Room & Facility Management Subsystem

Without a centralized, real-time inventory system, administrators face chronic issues with "ghost vacancies" (rooms that are empty but not listed) or double-bookings, where two tenants are promised the same unit. Furthermore, the physical condition of a room is often tracked separately from its availability, leading to situations where a tenant is assigned a room with broken amenities. This subsystem addresses these gaps by managing the comprehensive room inventory, including dynamic attributes like price, category, capacity, and current condition. The scope includes real-time availability tracking, handling boarding requests from tenants, and the ability to toggle status (e.g., locking a room as "Under Maintenance" to prevent bookings). Integration with architectural blueprints or asset depreciation schedules is out of scope.

- Display real-time room availability, occupancy status, and pricing.
- Manage room conditions, categories, and capacity details.
- Record facility issues such as broken fixtures or required repairs.
- Provide alerts for rooms marked as unavailable or under maintenance.

C. Billing & Payment Subsystem

Manual calculation of monthly dues combining fixed rent with variable utility readings is a primary source of friction, leading to calculation errors, disputes with tenants, and delayed cash flow. Additionally, tracking payments via screenshots of bank transfers or physical cash receipts is chaotic and makes financial reconciliation nearly impossible. This subsystem automates the entire billing chain: calculating total dues based on utility inputs, generating professional digital invoices, recording payments (supporting both manual cash entry and digital proofs), maintaining a transparent tenant ledger, and issuing downloadable receipts. Direct banking integration (automated fund transfer) and tax filing are out of scope; the system focuses on the internal recording of these transactions.

- Automatically compute monthly rent and utility charges.
- Record payments and generate receipts digitally.
- Track unpaid balances, overdue accounts, and payment history.
- Generate monthly financial summaries for management review.

D. Maintenance & Complaint Management Subsystem

In the current setup, tenant complaints are often verbal or sent via personal text messages to the Landlord, leading to a lack of accountability, forgotten requests, and no ability to track which facilities break down most often. This subsystem formalizes the support process. It includes a ticketing engine where tenants or admins can submit requests, a workflow board to track status (Pending to In Progress to Resolved), task assignment features for maintenance personnel, and an analytics module to identify recurring issues. The scope excludes inventory management of spare parts and external contractor payroll.

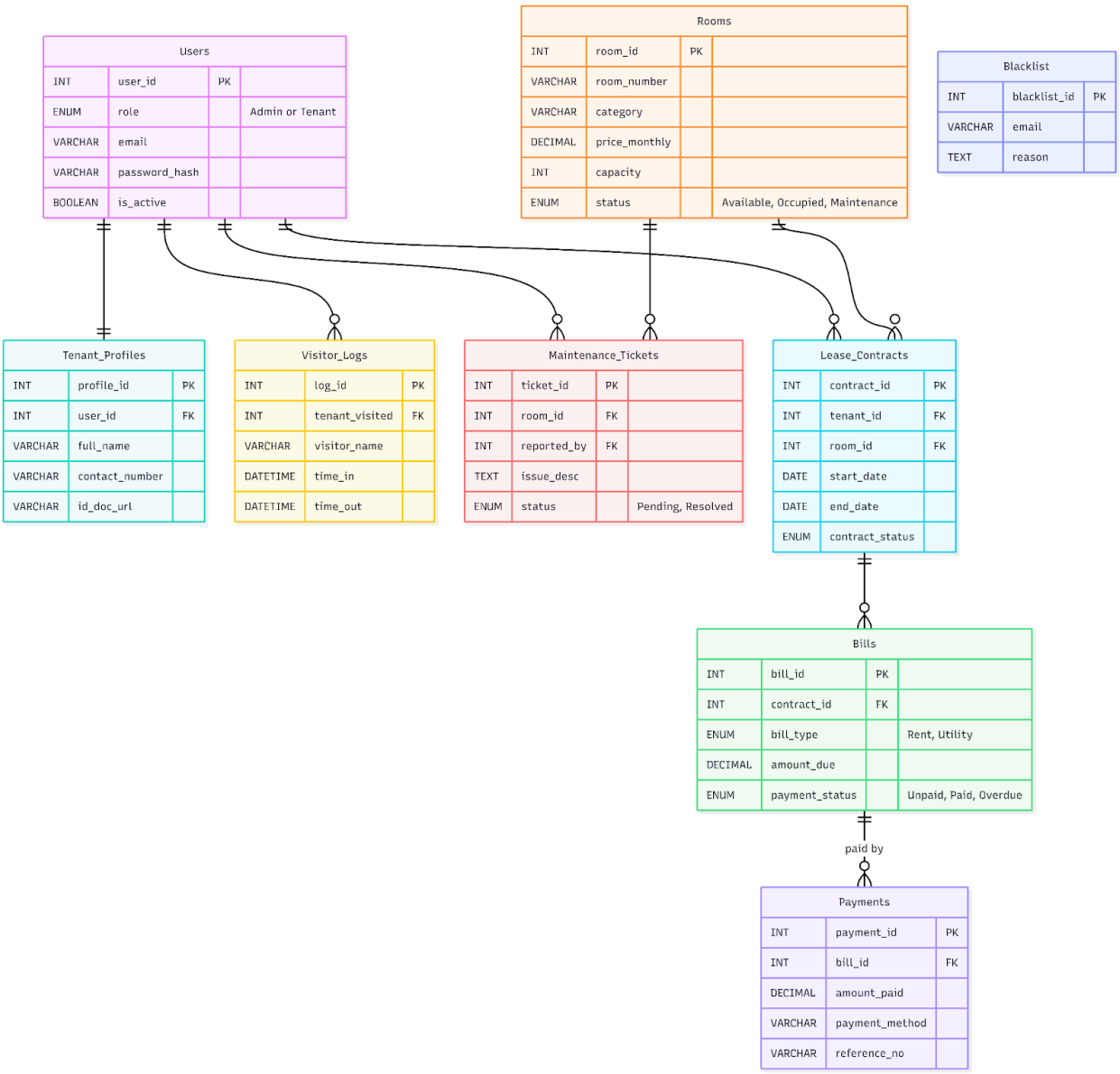
- Allow tenants to submit maintenance requests digitally.
- Log complaints, categorize them, and assign them for resolution.
- Track the progress and status of maintenance activities.
- Generate reports of recurring issues to help improve facility upkeep.

E. Security & Access Control Subsystem

Paper-based visitor logbooks are a security vulnerability; they are easily damaged, illegible, and impossible to search quickly during an investigation. Furthermore, enforcing a "Blacklist" is manual and prone to human error, allowing banned individuals to re-enter. This subsystem digitizes the entry/exit process. The scope covers digital logging of visitor details (Name, Purpose, Host, Time-In/Out), a searchable historical database, an incident reporting module for rule violations, and a strict Blacklist enforcement mechanism. Biometric hardware integration is out of scope; this is a software-based logging tool.

- Maintain a visitor log with timestamps, purpose, and who they visited.
- Record incidents or security-related concerns within the premises.
- Track access and room entry permissions if expanded in future upgrades.
- Provide management with oversight of movement and security activity.

Entity Relationship Diagram



Potential Challenges:

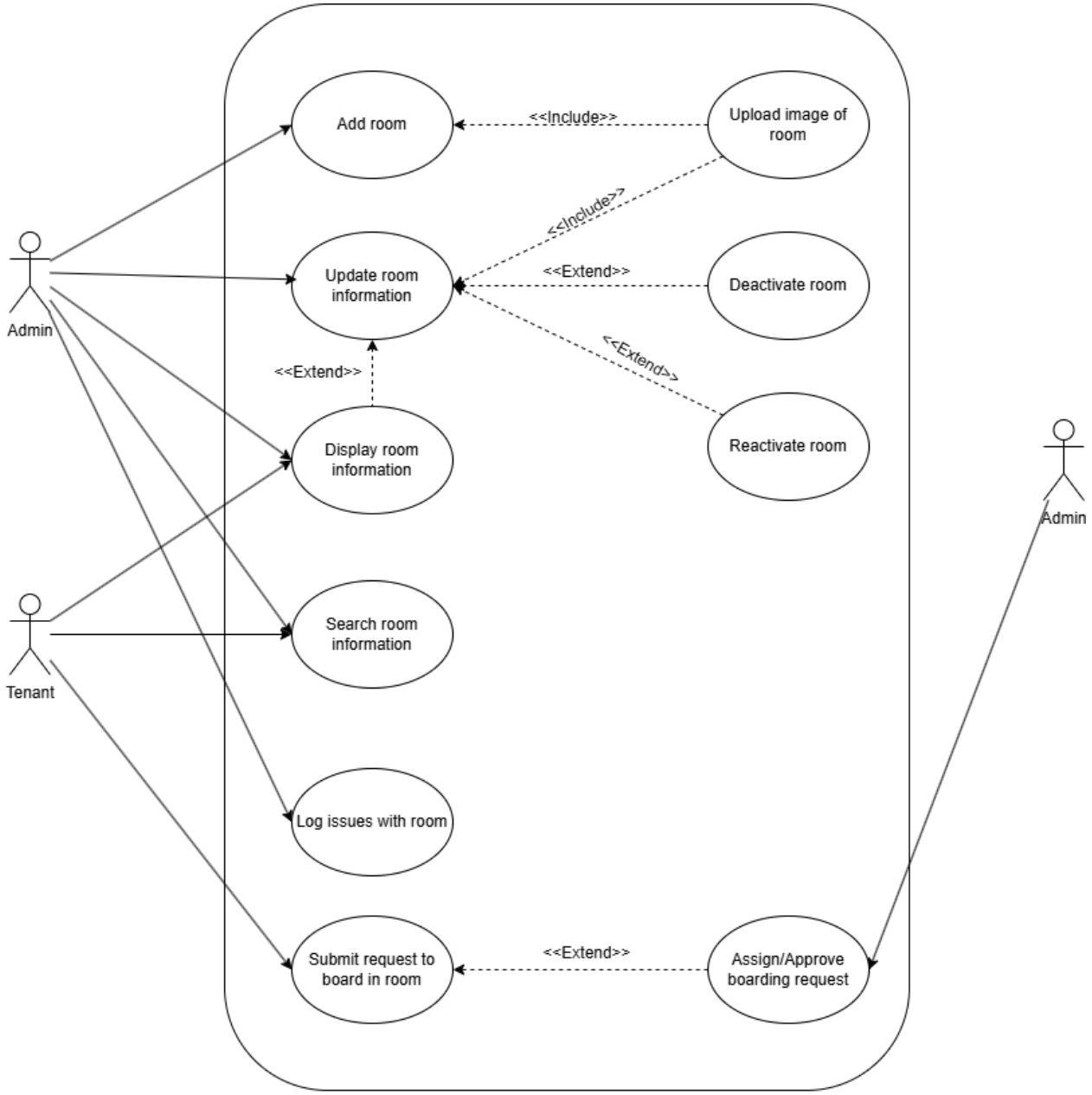
One of the technical challenges that may come up during the development of the system would most likely be the large scale and testing that needs to be done to assure the security of the system due to its multiple subsystem architecture. There is also the problem of integration of

web frameworks, which as of now, are still being decided. However, the leading framework in the mind of the developer is the Laravel Framework paired with Vue or React.

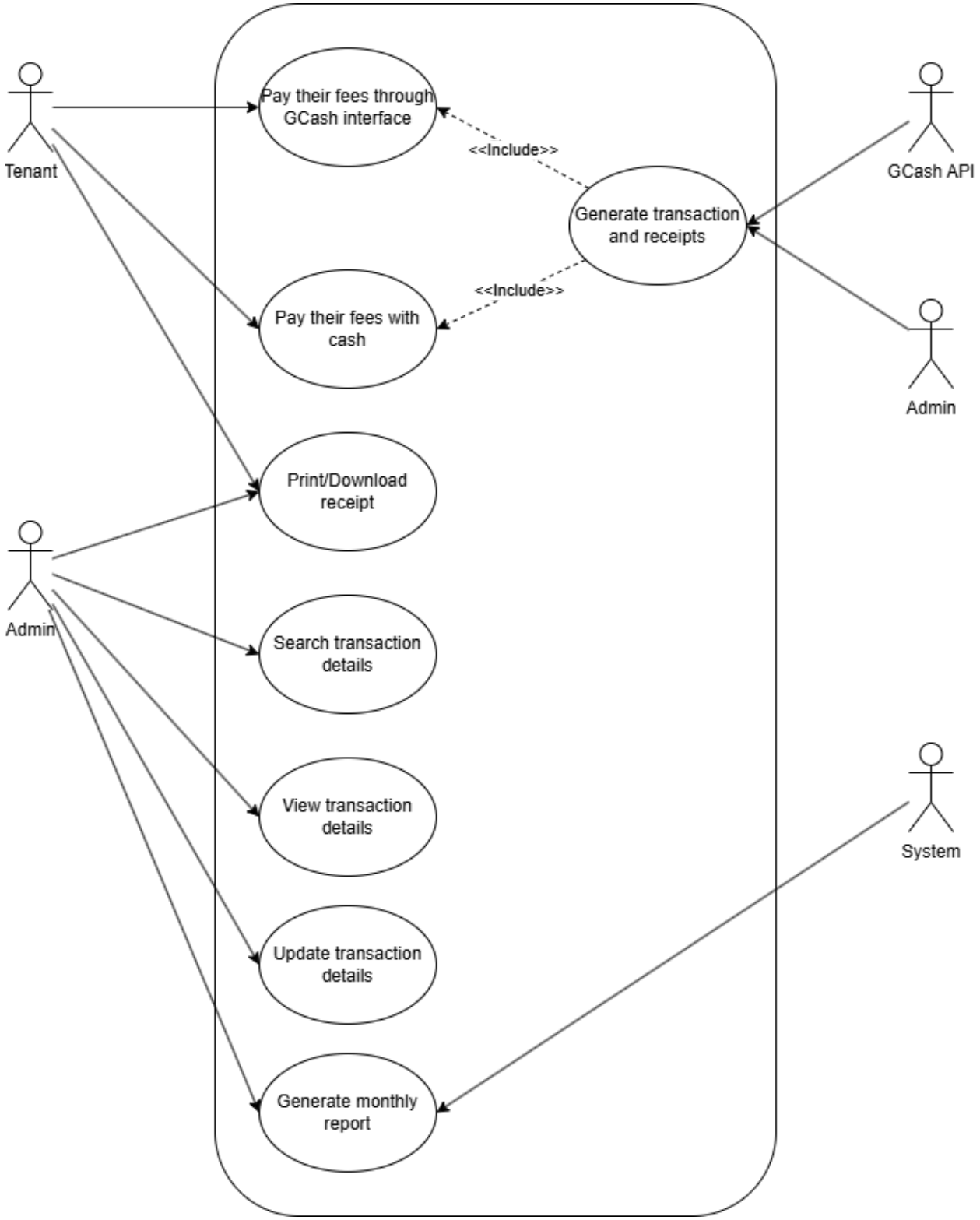
Use Case Diagrams

A. Tenant Management System

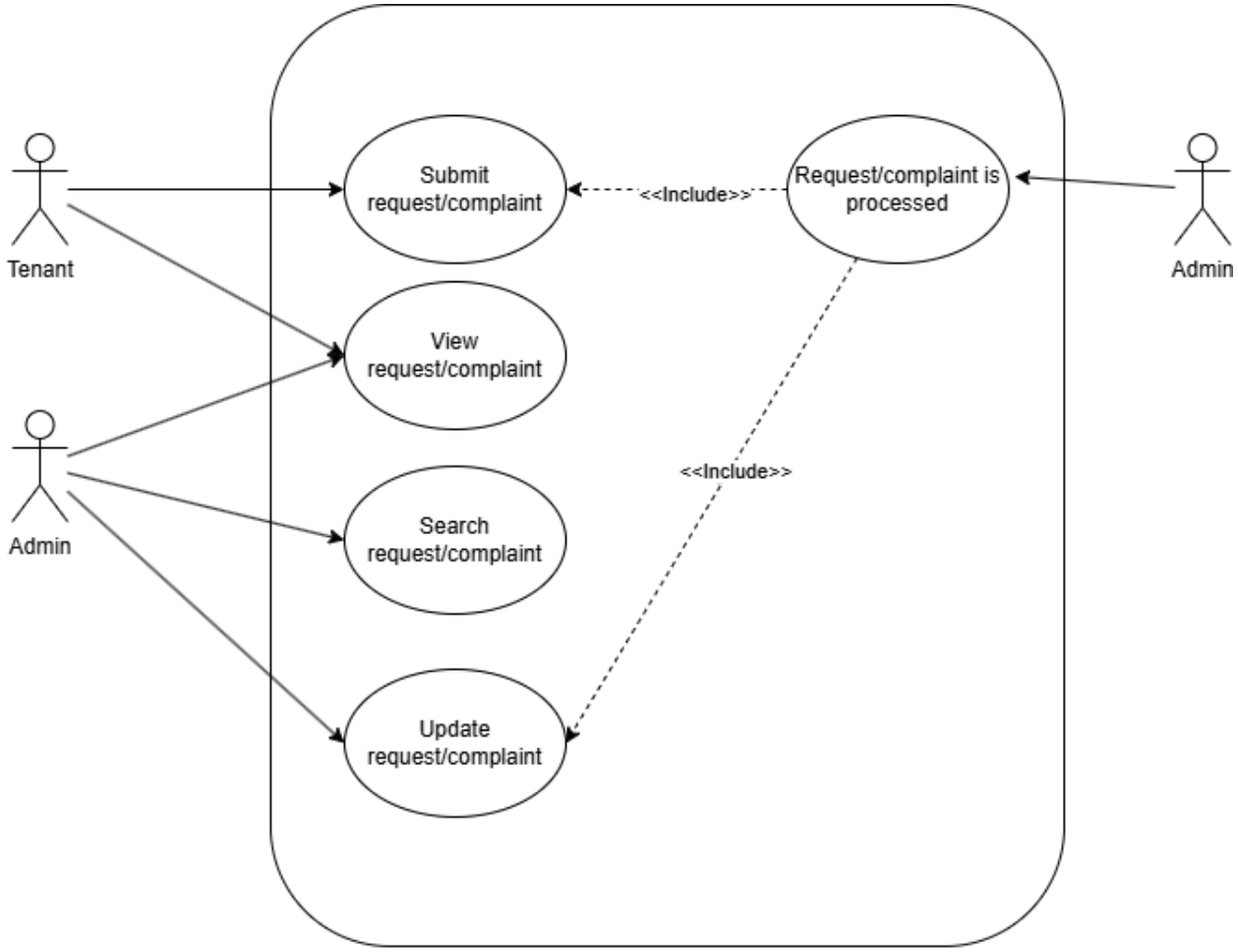
B. Room Management System



C. Billing and Payment System



D. Maintenance and Complaint Management System



E. Security and Access Control System

Data Dictionary and Profiles

A. Tenant Management System

Users Table

Field Name	**Data Type**	**Constraints**	**Description**
user_id	INT	PK, Auto Increment	Unique system identifier for the user.
role	ENUM	Not Null	User type: 'Admin' (Owner) or 'Tenant'.
email	VARCHAR(100)	Unique, Not Null	Email address used for login and banning identification.
password_hash	VARCHAR(255)	Not Null	Securely encrypted password string.
is_active	BOOLEAN	Default TRUE	TRUE = Active account; FALSE = Banned or archived.
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Date and time the account was registered.

Tenant Table

Field Name	Data Type	Constraints	Description
profile_id	INT	PK, Auto Increment	Unique identifier for the profile record.
user_id	INT	FK (Users), Not Null	Links this profile to a specific User account.
full_name	VARCHAR(150)	Not Null	The tenant's legal full name.
contact_number	VARCHAR(20)	Not Null	Primary mobile/contact number.
id_doc_url	VARCHAR(255)	Nullable	File path or URL to the uploaded ID document image.
emergency_contact	VARCHAR(150)	Nullable	Name and phone number of a contact in case of emergency.

Entity and Core Profiles

Entity Name	**Table Name**	**Description**	**Key Attributes**
User Account	Users	Central authentication table for both Owner (Admin) and Tenants.	user_id (PK), role (Admin/Tenant), email, password_hash, is_active.
Tenant Profile	Tenant_Profiles	Extension of the User table containing personal data specific to tenants.	profile_id (PK), user_id (FK), full_name, contact_number, id_doc_url.

B. Room Management System

Room Tables

Field Name	Data Type	Constraints	Description
room_id	INT	PK, Auto Increment	Unique identifier for the room.
room_number	VARCHAR(10)	Unique, Not Null	The physical label on the door (e.g., "101", "A-2").
category	VARCHAR(50)	Not Null	Classification (e.g., "Single", "Double").
price_monthly	DECIMAL(10,2)	Not Null	The base monthly rent cost for this room.
capacity	INT	Not Null	Maximum number of occupants allowed.
status	ENUM	Default 'Available'	Current state: 'Available', 'Occupied', or 'Maintenance'.
amenities	TEXT	Nullable	Comma-separated list or JSON of features (e.g., "WiFi, Bed").

Lease_Contracts Table

Field Name	Data Type	Constraints	Description
contract_id	INT	PK, Auto Increment	Unique identifier for the contract.
tenant_id	INT	FK (Users), Not Null	The tenant associated with this lease.
room_id	INT	FK (Rooms), Not Null	The room being rented.
start_date	DATE	Not Null	The date the lease officially begins.
end_date	DATE	Not Null	The date the lease is set to expire.
security_deposit	DECIMAL(10,2)	Default 0.00	Amount held by the owner as a deposit.
contract_status	ENUM	Default 'Active'	Lifecycle: 'Active', 'Pending_MoveOut', 'Terminated'.
move_out_req_date	DATE	Nullable	The date the tenant submitted a request to move out.

Entities and Core Profiles

Entity Name	**Table Name**	**Description**	**Key Attributes**
Room	Rooms	Represents physical rooms available for rent.	room_id (PK), room_number, price_monthly, status (Available/Occupied), amenities.
Lease Contract	Lease_Contracts	The legal agreement linking a Tenant to a Room for a specific duration.	contract_id (PK), tenant_id (FK), room_id (FK), start_date, end_date, contract_status.

C. Payment Management System

Bills Table

Field Name	Data Type	Constraints	Description
bill_id	INT	PK, Auto Increment	Unique identifier for the bill.
contract_id	INT	FK (Lease_Contracts)	Links the bill to a specific active lease.
bill_type	ENUM	Not Null	Category: 'Rent', 'Utility', 'Repair', 'Misc'.
description	VARCHAR(255)	Nullable	Brief details (e.g., "October Rent", "Faucet Replacement").
amount_due	DECIMAL(10,2)	Not Null	The total amount the tenant needs to pay.
due_date	DATE	Not Null	The deadline for payment.
payment_status	ENUM	Default 'Unpaid'	Status: 'Unpaid', 'Paid', 'Overdue', 'Waived'.
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Date the bill was generated.

Payments Table

Field Name	Data Type	Constraints	Description
payment_id	INT	PK, Auto Increment	Unique identifier for the payment transaction.
bill_id	INT	FK (Bills), Not Null	The specific bill this payment is settling.
amount_paid	DECIMAL(10,2)	Not Null	The actual amount of money received.
payment_method	VARCHAR(20)	Not Null	Method used: 'Cash', 'GCash', 'Bank Transfer'.
reference_no	VARCHAR(50)	Nullable	Transaction reference number (for GCash/Bank) or Receipt #.
receipt_url	VARCHAR(255)	Nullable	File path to the generated digital receipt image.
payment_date	TIMESTAMP	Default CURRENT_TIMESTAMP	Date and time the payment was recorded.

Entities and Core Profiles

Entity Name	**Table Name**	**Description**	**Key Attributes**
Bill	Bills	A record of debt (money owed) generated by the system/admin.	bill_id (PK), contract_id (FK), bill_type (Rent/Utility), amount_due, payment_status.
Payment	Payments	A record of credit (money received) from the tenant.	payment_id (PK), bill_id (FK), amount_paid, payment_method (GCash/Cash), reference_no.

D. Maintenance and Complaint Management System

Maintenance_Tickets Table

Field Name	**Data Type**	**Constraints**	**Description**
ticket_id	INT	PK, Auto Increment	Unique identifier for the ticket.
room_id	INT	FK (Rooms), Nullable	The room where the issue exists (Null if common area).
reported_by	INT	FK (Users), Not Null	The user (Tenant or Admin) who reported the issue.
issue_desc	TEXT	Not Null	Detailed description of the maintenance problem.
priority	ENUM	Default 'Medium'	Urgency: 'Low', 'Medium', 'High'.
status	ENUM	Default 'Pending'	Progress: 'Pending', 'In Progress', 'Resolved'.
contractor_notes	TEXT	Nullable	Admin's notes on the fix (e.g., "Fixed by Mario").

created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Date the issue was reported.
resolved_at	TIMESTAMP	Nullable	Date the issue was marked as resolved.

Entities and Core Profiles

Entity Name	**Table Name**	**Description**	**Key Attributes**
Maintenance Ticket	Maintenance_Tickets	Reports of broken items or complaints.	ticket_id (PK), room_id (FK), reported_by (FK), status (Pending/Resolved), contractor_notes.

E. Security and Access Control System

Visitor Table

Field Name	**Data Type**	**Constraints**	**Description**
log_id	INT	PK, Auto Increment	Unique identifier for the log entry.
tenant_visited	INT	FK (Users), Not Null	The tenant who is receiving the guest.
visitor_name	VARCHAR(100)	Not Null	Name of the visitor.
purpose	VARCHAR(255)	Nullable	Reason for the visit.
time_in	DATETIME	Default CURRENT_TIMESTAMP	Time the visitor arrived.
time_out	DATETIME	Nullable	Time the visitor departed.

Blocklist Table

Field Name	Data Type	Constraints	Description
blacklist_id	INT	PK, Auto Increment	Unique identifier for the blacklist record.
email	VARCHAR(100)	Unique, Not Null	The email address banned from the system.
reason	TEXT	Not Null	The reason for the ban (e.g., "Repeated Non-payment").
banned_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Date the ban was enacted.

Entities and Core Profiles

Entity Name	Table Name	Description	Key Attributes
Visitor Log	Visitor_Logs	Tracks non-residents entering the premises.	log_id (PK), tenant_visited (FK), visitor_name, time_in, time_out.
Blacklist	Blacklist	Security list to prevent banned individuals from registering.	blacklist_id (PK), email, reason, banned_at.